ST JOSEPH ENGINEERING COLLEGE MANGALURU

AN AUTONOMOUS INSTITUTION

The Department of Electronics And Communication

Course Level Problem Based Learning

Course Title: DDHDL Verilog

Course Code:22ECE42

Semester: Fourth Semester

Project Report Titled

FPGA implementation of ECG signals using FIR filter

Submitted By

Sl No.	USN	Name
1	4SO24EC006	AJAY KUMAR P
2	4S024EC018	ASTEL JOSWIN RODRIGUES
3	4S024EC021	BASHMA
4	4SO24EC022	BRAHMI LAKSHMI U.H

Under the Guidance of

P.C Vijay Ganesh

(Assistant Professor, B.E, M. Tech, MIEEE)

PBL SECONDARY RESEARCH REPORT ON

FPGA implementation of ECG signals using FIR filter

Chapter 1: INTRODUCTION:

Electrocardiogram (ECG) signal processing is a fundamental aspect of modern biomedical engineering, serving as a primary diagnostic tool for monitoring cardiac health. As wearable health technology and real-time patient monitoring systems become more prevalent, the need for high-speed, hardware-efficient signal processing architectures has increased. Implementing digital filters on Field Programmable Gate Arrays (FPGAs) provides a robust solution for these requirements, offering a balance between the flexibility of software and the high-speed performance of dedicated hardware

1.1 Background:

An ECG signal captures the electrical activity of the heart over time using electrodes placed on the skin. These signals are inherently low-frequency and low-amplitude, making them highly susceptible to various forms of noise and interference. To extract a clean signal for clinical analysis, digital filtering is essential. Finite Impulse Response (FIR) filters are widely preferred in medical applications because they are inherently stable and provide a linear phase response, which ensures that the different frequency components of the ECG signal are delayed by the same amount, preventing distortion of the P-QRS-T wave morphology.

1.2 Problem Statement:

software-based ECG processing (on PCs or microcontrollers) is common, it often lacks the throughput and parallel processing capabilities required for multi-channel, real-time monitoring in high-stakes environments. Traditional Digital Signal Processors (DSPs) may also struggle with power consumption constraints in portable medical devices. Noise in ECG signals can mask critical cardiac features, leading to misdiagnosis. There is a specific challenge in designing a filter that can effectively suppress wideband noise and power line interference without sacrificing the diagnostic integrity of the heartbeat's

delicate peaks and intervals.

1.3 Objectives:

The primary goal of this project is to design and implement a high-performance FIR filter on an FPGA platform to enhance the quality of ECG signals. Filter Design & Simulation**:** To determine the optimal filter coefficients, leaf cells to achieve the desired frequencies for denoising ECG data. Hardware Description: To model the FIR filter architecture using VHDL or Verilog, focusing on efficient resource utilization (multipliers and adders). FPGA Implementation: To synthesize and implement the design on an FPGA board, utilizing its parallel processing capabilities to act as an accelerator for real-time signal filtration**.** Performance Evaluation: To validate the filtered output against the original noisy signal, ensuring high Signal-to-Noise Ratio (SNR) improvement and minimal signal distortion.

Chapter 2: Literature Review (Secondary Research)

2.1 Review of Existing Work:

1) As per the literature survey done by our team, we have an overview of top four research papers. Each study contributes a distinct methodological perspective, ranging from median filtering and Kaiser-windowed FIR design to Montgomery multiplier-accelerated adaptive filtering and Bessel-based lowpass filter synthesis.

In paper 1 published by Raykar and Bansod proposed a real-time ECG denoising architecture implemented on the Xilinx Spartan 3E FPGA using Verilog HDL. The system exploits the spike-removal technique borrowed from audio signal processing. The work demonstrates a significant reduction in median filter window size (91 versus 501 in the prior MATLAB-based work at 1000 samples/s), thereby reducing BRAM requirements and improving throughput. However, the authors do not report FPGA resource utilization metrics such as LUT count, flip-flip usage, or maximum clock frequency, which limits direct comparison with other architectures.

2) In paper 2 published by Uttraphan presented a comprehensive hardware implementation study of FIR filters for ECG signal denoising on the Intel Cyclone IV FPGA. Three distinct designs were implemented and compared. Design 1 uses direct hardware mapping with 21 parallel multipliers, achieving a maximum operating frequency of 50.66 MHz This design consumes 2,646 logic elements (42% of Cyclone IV resources) and dissipates 93.47 mW. Design 2 applies operation scheduling with a single multiplier controlled by an Arithmetic State Machine (ASM), reducing logic elements to just 145 (2% utilization) and power to 74.06 mW,). Design 3 uses three multipliers with concurrent computation, achieving 2,500 µs latency with 290 logic elements and 73.49 mW. The parallel and pipelined hardware architecture inherent to FPGA implementation is shown to decisively outperform software despite operating at a much lower clock frequency (50 MHz vs. 3.2 GHz), demonstrating the efficiency advantage of hardware parallelism.

3) In paper 2 published by Mulla and Kasetwar addressed the hardware implementation of an adaptive Least Mean Squares (LMS) FIR filter for ECG noise cancellation, with the key innovation of replacing the conventional multiplication unit with an efficient Radix-2 Montgomery multiplier.

The Montgomery multiplier architecture requires only 197 LUTs/slices—significantly fewer than prior implementations such as the Xilinx Vertex II linear systolic design (4,821 LUTs at 243.9 MHz). The system achieved a 22 dB SNR improvement from input to output across 10 test signal samples. The paper provides a compelling case for Montgomery multiplication as a hardware-efficient alternative in adaptive DSP systems, particularly where iterative weight updates demand high-throughput multiplication. Limitations include the restriction to 8-bit signals and only 2 filter taps, which may constrain the achievable noise rejection in complex ECG artifact scenarios requiring higher-order adaptive models

4) paper 4 describes the design and FPGA co-simulation of a Bessel Lowpass Digital FIR Filter targeting a 6 µs pulse wave for Medical Linear Accelerator (LINAC) applications. The device utilization summary shows 128 slice flip-flops (1%), 159 four-input LUTs (1%), 6 MULT18X18SIOs (30%), and 51 bonded IOBs (13%). The critical path delay (maximum clock delay) is 1.713 ns. Filter order selection was driven by comparison across orders 2, 6, 8, 10, and 15, with order

15 providing the best balance between pulse fidelity and implementation stability. While the application domain differs from ECG denoising, the underlying FIR filter architecture and co-simulation methodology are directly applicable to biomedical ECG processing pipelines.

2.2 Comparative Analysis Table

Paper	Method	FPGA Target	Key Metric	SNR/Speed	HDL Used
Raykar & Bansod (2015)	Dual Median Filter	Xilinx Spartan 3E	Window size 91+7	SNR –10.33 dB	Verilog
Uttraphan et al. (2024)	FIR – Kaiser Window	Intel Cyclone IV	21-tap, 50 MHz	1,250× faster	Verilog
Mulla & Kasetwar (2014)	Montgomery LMS Adaptive	Xilinx XC3S500E	Radix-2, 113 MHz	SNR +22 dB	VHDL
ICNTE 2021 (Bessel FIR)	Bessel LPF order-15	Xilinx Spartan 6A	Max delay 1.713 ns	N/A	VHDL

2.3 Research Gap Identification

Limited resource utilization reporting: most of papers didn't report any FPGA resource utilization metrics (LUT count, flip-flop usage, maximum operating frequency, or power consumption). Without these figures, the design cannot be benchmarked against modern implementations or ported to contemporary FPGA families. Similarly, the ICNTE 2021 paper reports utilization for the Spartan 6A but does not characterize throughput in terms of samples per second or processing latency for a full ECG episode. Standardized reporting of

at minimum Fmax, total logic elements, power dissipation, and throughput is required for fair comparative evaluation.

Absence of real time streaming validation: All reviewed implementations process pre-stored ECG records loaded from memory (MIT-BIH database files in ROM, or Physionet data). None demonstrates real-time streaming operation with live sensor data—i.e., acquisition from an analog front-end, ADC interface, streaming through the filter pipeline, and continuous output. Integration with an ADC interface (SPI or I2C protocol controller) and a real-time output display (UART to PC or LCD) is absent across all papers, leaving an important gap between laboratory demonstration and deployable clinical prototype.

Power consumption for wearable deployment: For the intended application of wearable or implantable ECG monitoring, where battery capacity is severely constrained, sub-milliwatt operation is desirable. None of the reviewed papers targets low-power FPGA families (e.g., Xilinx Artix-7 with dynamic power management, or Lattice iCE40 UltraPlus for ultra-low-power IoT applications), and none employs clock gating, operand isolation, or dynamic voltage-frequency scaling—established techniques for FPGA power reduction. This gap is particularly significant given the rapid growth of continuous ambulatory cardiac monitoring.

Chapter 3: Proposed System Design

3.1 Design Justification

1. What is the reason for selecting a 16-tap filter?

A 16-tap FIR filter is selected to achieve effective noise removal while maintaining reasonable hardware complexity. It provides a good balance between filtering performance and resource utilization in FPGA implementation.

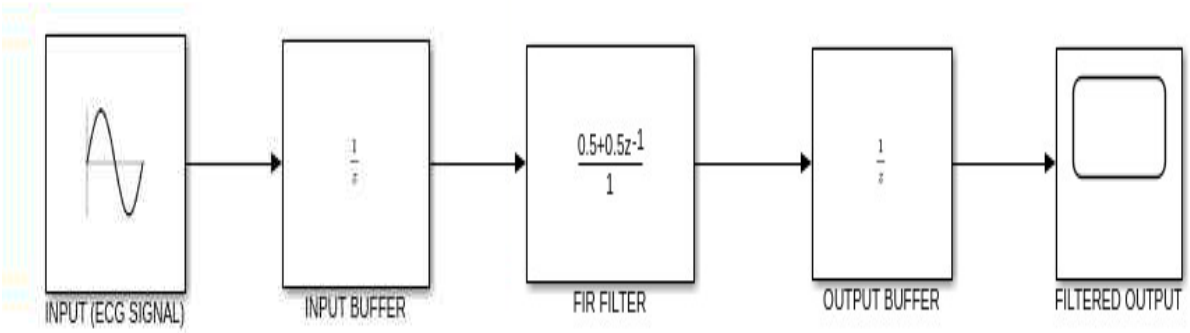
2. What are the limitations of using fewer taps?

Using fewer taps reduces hardware usage but leads to poor filtering performance. It may not effectively remove noise and can distort important ECG signal features.

3.What are the drawbacks of using more taps than required?

Increasing the number of taps improves filtering accuracy but significantly increases hardware complexity, power consumption, and processing delay. This makes the design less efficient for real-time applications.

3.2 Block Diagram



ECG Input

The ECG input provides the raw biomedical signal which may contain useful cardiac information along with noise and interference.

FIR Filter Core

The FIR filter core is the main processing unit of the system. It performs filtering by applying predefined coefficients to the input signal. The filter consists of delay elements, multipliers, and adders that together perform convolution to remove noise from the ECG signal.

Filtered ECG Output

The output of the system is the filtered ECG signal with reduced noise, making it suitable for further analysis and interpretation.

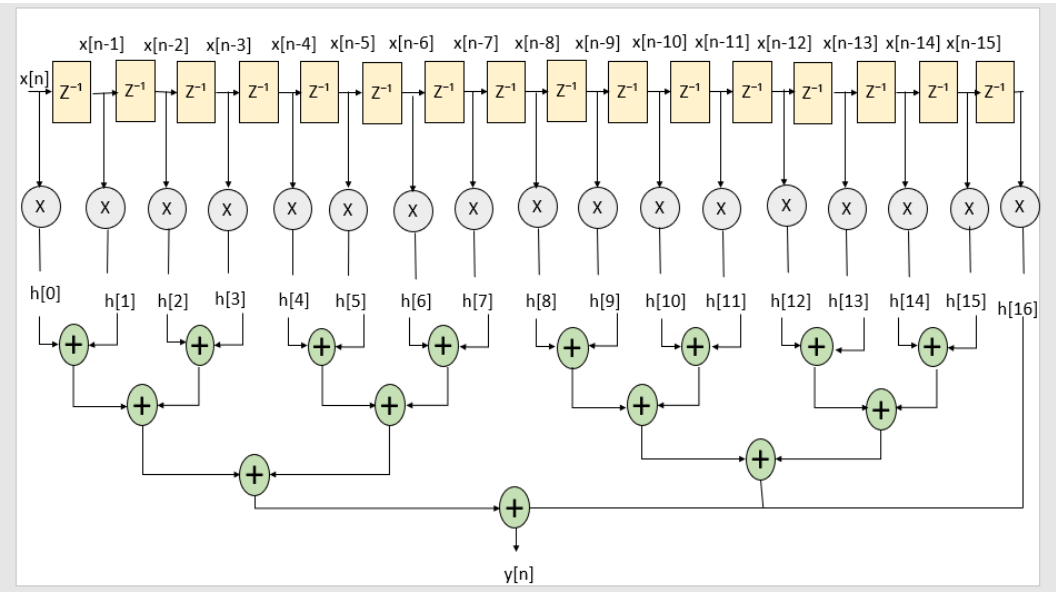
Input Buffer

The input buffer temporarily stores incoming sampled ECG signals before they are processed by the FIR filter. It helps in providing past and present input values required for convolution and ensures smooth data flow into the system.

Output Buffer

The output buffer holds the processed (filtered) ECG signal after computation. It stores the result before sending it to the next stage or display, ensuring proper timing and stable output transfer.

3.3 Architecture Diagram



$$y(n)=h(0)x(n)+h(1)x(n-1)+h(2)x(n-2)+h(3)x(n-3)+h(4)x(n-4)+h(5)x(n-5)+\ldots\ldots+h(15)x(n-15)+h(16)x(n-16)$$

Delay Block

Each delay block stores the previous input sample. In a 16-tap FIR filter, there are 15 delay blocks. These create past versions of the input signal required for filtering.

Multipliers

Each input (current and delayed samples) is multiplied with a fixed coefficient. These coefficients define the filter characteristics (low-pass, band-pass, etc.). A 16-tap filter uses 16 multipliers.

Coefficients (h[n])

These are fixed values stored in the system. They decide how much weight each input sample contributes to the final output of the FIR filter.

Adders

The outputs of all multipliers are added together. A 16-tap FIR filter requires 15 adders to combine all multiplied values into one final result.

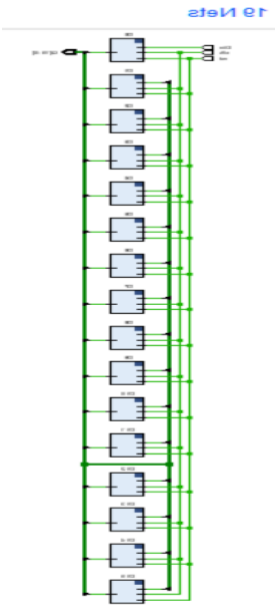
Chapter 4: Leaf Cell Design

4.1 Identification of Leaf Cells

- Adder
- Multiplier
- Delay

4.2 Leaf Cell Description

- Delay



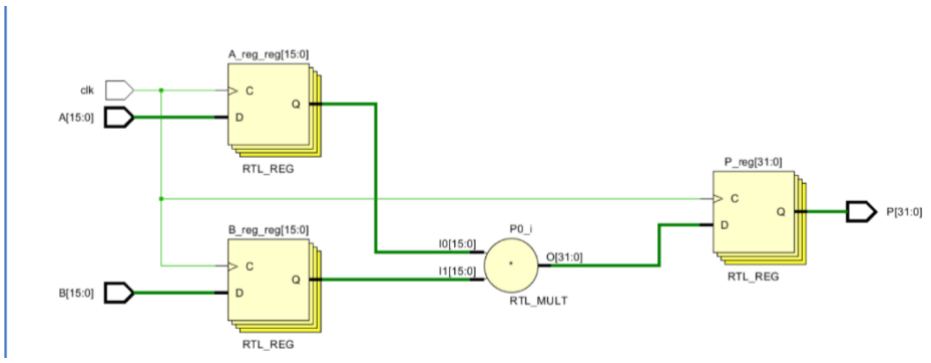
Functionality

The delay (leaf cell) is used to store and shift past input samples so that each sample can be multiplied with the correct filter coefficient. It works like a register or shift element, where the current ECG sample moves through a chain of delays, creating time-shifted versions . This is essential for FIR operation because the filter output depends on both present and previous samples. The delay ensures proper timing alignment, maintains data synchronization, and enables accurate convolution of the input signal with filter coefficients. It is typically implemented using flip-flops/registers, designed to be stable, fast, and clock-driven for reliable signal processing.

- **Optimization used**

Shift register implementation to reduce control complexity
Register sharing to save hardware
Efficient clocking to reduce power consumption

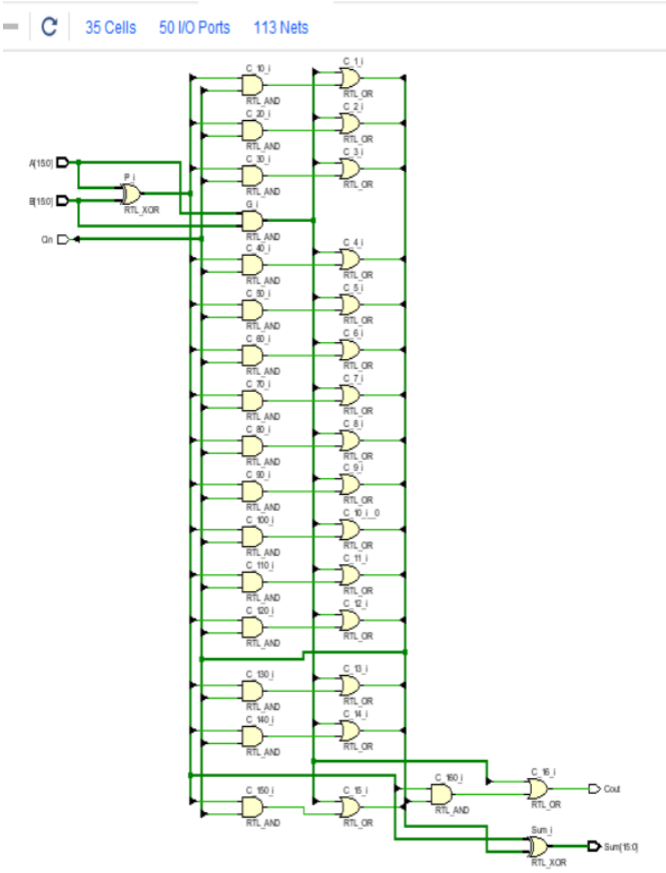
- **Multiplier**



This diagram shows a **16-bit pipelined multiplier design** where the inputs A [15:0] and B [15:0] are first stored in registers to ensure proper synchronization with the clock. These registered values are then fed into a multiplier block, which generates a 32-bit output. The result is further stored in an output register before being sent out as P[31:0]. This use of input and output registers makes the design faster and more stable by improving timing and data flow.

- Optimization used
  Booth's Algorithm to reduce number of partial products
  Use of pipelining to increase speed
  Utilization of DSP blocks (in FPGA) for efficient multiplication

- Adder



- **Functionality**

  the **adder (leaf cell)** performs the basic operation of **summing the outputs of all multipliers (tap results)**. Each multiplier produces a partial product of the ECG sample and filter coefficient, and the adder combines these values step-by-step to generate the final filtered output. The adder ensures **accurate accumulation**, supports **signed arithmetic** (since ECG signals can be positive and negative), and helps maintain the **signal integrity** without distortion. It is designed to be **fast and efficient**, as multiple additions are required in FIR filtering, and it may be used in structures like **adder trees or accumulators** to reduce delay and improve performance.

- Optimization used

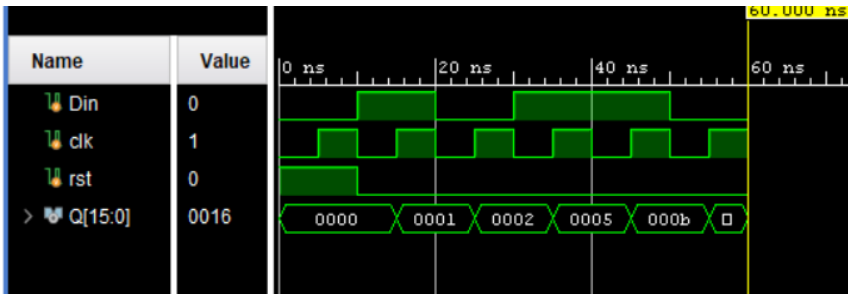
  4-bit Adder for faster operation

  Reduced propagation delay

  Bit-width optimization to minimize hardware usage

4.3 Leaf Cell Simulation

- **Delay**



- **Input test cases**

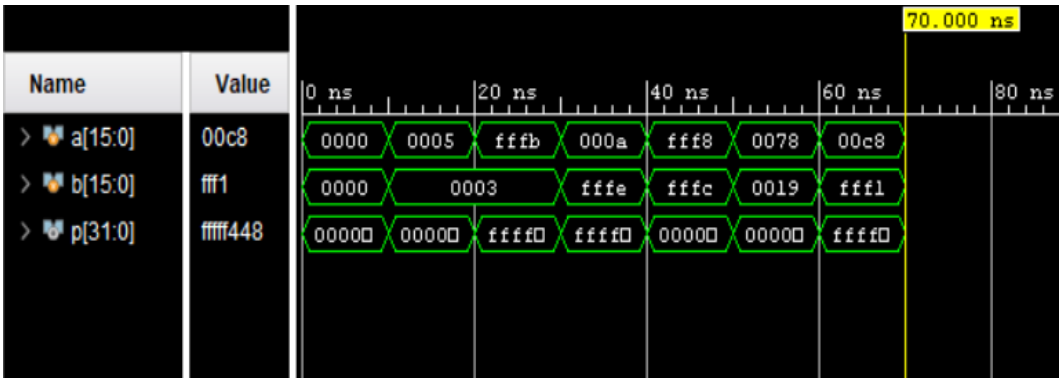
  the delay input starts with initializing the system where the clock is set to 0, reset is enabled (set to 1), and input (Din) is 0 for the first 10 time units, which clears the shift register. After that, the reset is disabled (reset = 0), and a sequence of input bits is applied to Din at intervals of 10 time units. The input sequence provided is 1, 0, 1, 1, 0, where each bit is fed into the shift register on each clock cycle. Meanwhile, the clock continues to toggle every 5 time units, ensuring that data is shifted properly on each rising (or active) clock edge.

- **Output verification**

Time	Din	Q (16-bit Output)
0	0	0000 0000 0000 0000
10	1	0000 0000 0000 0001
20	0	0000 0000 0000 0010
30	1	0000 0000 0000 0101

Time	Din	Q (16-bit Output)
40	1	0000 0000 0000 1011
50	0	0000 0000 0001 0110

- **Multiplier**



- **Input test cases**

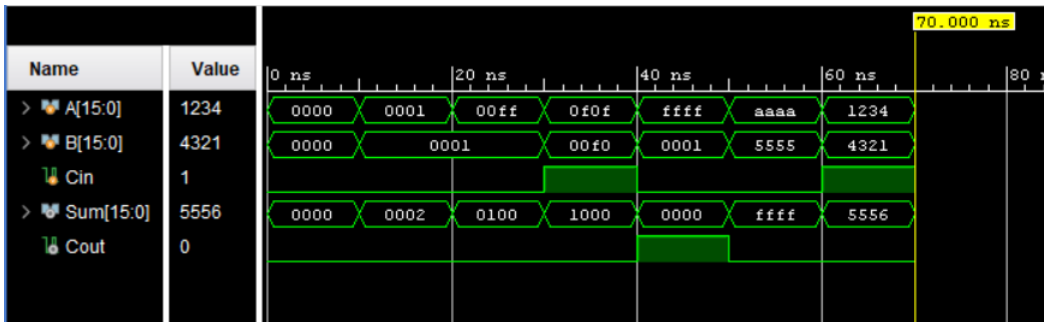
  The testbench applies a sequence of signed input values to the multiplier over time. Initially, both inputs are set to **a = 0** and **b = 0**. After 10 ns, the inputs change to **a = 5** and **b = 3**. In the next interval, the values become **a = -5** and **b = 3**, followed by **a = 10** and **b = -2**. Then the inputs are **a = -8** and **b = -4**, testing multiplication of two negative numbers. After that, larger values are applied with **a = 120** and **b = 25**, and finally **a = 200** and **b = -15**. These test cases collectively verify the multiplier's functionality for zero, positive numbers, negative numbers, and mixed-sign operations.

- Output verification

Time (ns)	a	b	Expected Output p
0–10	0	0	0
10–20	5	3	15

Time (ns)	a	b	Expected Output p
20–30	-5	3	-15
30–40	10	-2	-20
40–50	-8	-4	32
50–60	120	25	3000
60–70	200	-15	-3000

- **Adder**



- **Input test cases**

  The input test cases in this 16-bit adder testbench are applied by varying the operands A, B, and the carry-in Cin to check different addition conditions. Initially, both inputs are set to zero with no carry-in (A = 0000, B = 0000, Cin = 0) to verify the basic case. Then small values are added (A = 0001, B = 0001, Cin = 0) to check simple addition. Next, a carry generation case is tested using A = 00FF and B = 0001, which produces a carry within the lower byte. The testbench then applies mixed values with carry-in enabled (A = 0F0F, B = 00F0, Cin = 1) to verify carry propagation. A maximum value case is tested with A = FFFF and B = 0001 to check overflow and final carry-

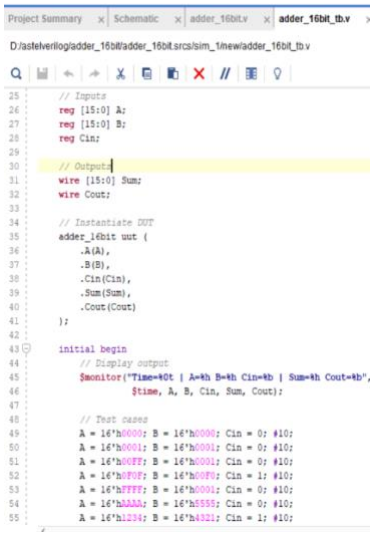
out. Another case uses alternating bit patterns (A = AAAA, B = 5555, Cin = 0) to validate bitwise addition across all positions. Finally, random values with carry-in (A = 1234, B = 4321, Cin = 1) are applied to ensure general correctness. Each test case is applied sequentially with a delay of 10 time units.

- Output verification

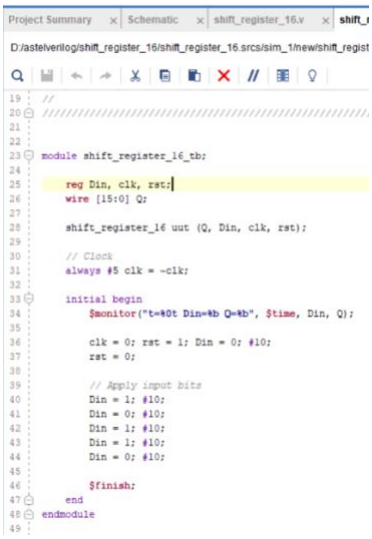
	A	B	Cin	Sum	Cout
0	0000	0000	0	000 0	0
10	0001	0001	0	0002	0
20	00FF	0001	0	0100	0
30	0F0F	00F0	1	1000	0
40	FFFF	0001	0	0000	1
50	AAAA	5555	0	FFFF	0
60	1234	4321	1	5556	0

Appendix

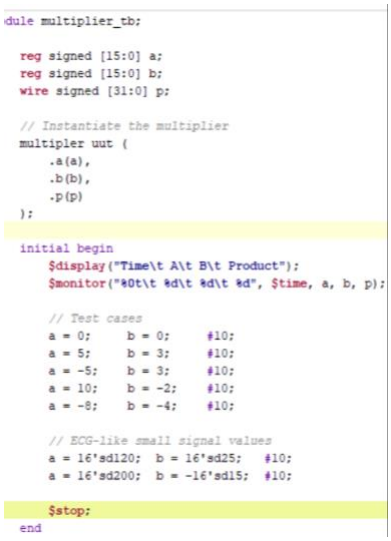
● Simulation screenshots



```
Project Summary | Schematic | adder_16bit.v | adder_16bit_tb.v
D:/astelverilog/adder_16bit/adder_16bit.srcs/sim_1/new/adder_16bit_tb.v

    // Inputs
    reg [15:0] A;
    reg [15:0] B;
    reg Cin;

    // Outputs
    wire [15:0] Sum;
    wire Cout;

    // Instantiate DUT
    adder_16bit uut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );

    initial begin
        // Display output
        $monitor("Time=%0t | A=%h B=%h Cin=%b | Sum=%h Cout=%b",
                 $time, A, B, Cin, Sum, Cout);

        // Test cases
        A = 16'h0000; B = 16'h0000; Cin = 0; #10;
        A = 16'h0001; B = 16'h0001; Cin = 0; #10;
        A = 16'h00FF; B = 16'h0001; Cin = 0; #10;
        A = 16'h0F0F; B = 16'h00F0; Cin = 1; #10;
        A = 16'hFFFF; B = 16'h0001; Cin = 0; #10;
        A = 16'hAAAA; B = 16'h5555; Cin = 0; #10;
        A = 16'h1234; B = 16'h4321; Cin = 1; #10;
```



```
Project Summary | Schematic | shift_register_16.v | shift_r
D:/astelverilog/shift_register_16/shift_register_16.srcs/sim_1/new/shift_regist

//
////////////////////////////////////////////////////////////////////////////////

module shift_register_16_tb;

    reg Din, clk, rst;
    wire [15:0] Q;

    shift_register_16 uut (Q, Din, clk, rst);

    // Clock
    always #5 clk = ~clk;

    initial begin
        $monitor("t=%0t Din=%b Q=%b", $time, Din, Q);

        clk = 0; rst = 1; Din = 0; #10;
        rst = 0;

        // Apply input bits
        Din = 1; #10;
        Din = 0; #10;
        Din = 1; #10;
        Din = 1; #10;
        Din = 0; #10;

        $finish;
    end
endmodule
```



```
dule multiplier_tb;

  reg signed [15:0] a;
  reg signed [15:0] b;
  wire signed [31:0] p;

  // Instantiate the multiplier
  multipler uut (
      .a(a),
      .b(b),
      .p(p)
  );

  initial begin
      $display("Time\t A\t B\t Product");
      $monitor("%0t\t %d\t %d\t %d", $time, a, b, p);

      // Test cases
      a = 0;     b = 0;     #10;
      a = 5;     b = 3;     #10;
      a = -5;    b = 3;     #10;
      a = 10;    b = -2;    #10;
      a = -8;    b = -4;    #10;

      // ECG-like small signal values
      a = 16'sd120;  b = 16'sd25;   #10;
      a = 16'sd200;  b = -16'sd15;  #10;

      $stop;
  end
```

● Tool used

Vivado 2018.2

Simulink

power point

● References (IEEE format)

1) *Journal of Advanced Research in Applied Sciences and Engineering Technology*

Volume 40, Issue 2 (2024) 50-61

2) International Journal of Advanced Research in Electrical,

Electronics and Instrumentation Engineering

(An ISO 3297: 2007 Certified Organization)

Vol. 4, Issue 6, June 2015

3) FPGA Implementation of An Efficient Montgomery

Multiplier For Adaptive Filtering Application

4) FPGA Implementation of optimized FIR Filter For ECG denoising using Peres reversible logic gates.